

Market_Sentiment_Predictor_Pipeline

February 6, 2026

1 Market Sentiment Predictor

Geovanni Jones, Josh Previte, Casey Barrasso

The goal of this project is to predict the directional price movement of a specific stock using both historical price data and news information. Two separate models will predict price movements based on historical data and analyze news sentiment.

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import re

from collections import Counter

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import STL
```

```
[2]: os.getcwd()
```

```
[2]: '/home/jupyter-jonesaop/market-sentiment-predictor/notebooks'
```

2 Week 2 - Data Ingestion

There are two main datasets to this project. The first is a dataset containing news article content including titles, content, and sentiment scores from various news sources around the AAPL stock. The second dataset (introduced later in the notebook) contains a time series of AAPL stock prices. The final dataset will combine the two datasets on date.

2.1 Sentiment Dataset

```
[3]: #Github didn't allow us to upload a file more than 25MB. This is combining all
    ↳ files into one file
raw_data_1 = pd.read_csv('../data/apple_news_data_1.csv', encoding='latin-1')
raw_data_2 = pd.read_csv('../data/apple_news_data_2.csv', encoding='latin-1')
raw_data_3 = pd.read_csv('../data/apple_news_data_3.csv', encoding='latin-1')
```

```
raw_data_4 = pd.read_csv('../data/apple_news_data_4.csv', encoding='latin-1')
raw_data_5 = pd.read_csv('../data/apple_news_data_5.csv', encoding='latin-1')

raw_data = pd.concat([raw_data_1, raw_data_2, raw_data_3, raw_data_4,
↳raw_data_5])[['date', 'title', 'content', 'link', 'symbols', 'tags',
↳'sentiment_polarity', 'sentiment_neg', 'sentiment_neu', 'sentiment_pos']]
```

```
[4]: #view a small subsection of the data to ensure formatting looks correct
raw_data.head(10)
```

```
[4]:
```

	date \		title \		content \		link \
0	2021-04-30T21:38:00+00:00						
1	2021-04-30T20:41:52+00:00						
2	2021-04-30T20:18:00+00:00						
3	2021-04-30T19:32:30+00:00						
4	2021-04-30T18:49:00+00:00						
5	2021-04-30T18:30:06+00:00						
6	2021-04-30T18:19:10+00:00						
7	2021-04-30T18:13:06+00:00						
8	2021-04-30T17:54:31+00:00						
9	2021-04-30T17:34:59+00:00						

```

0 https://finance.yahoo.com/m/bbe3f7c9-c7af-3c9f...
1 https://finance.yahoo.com/news/valuation-remai...
2 https://finance.yahoo.com/m/6e441d74-37c1-3642...
3 https://finance.yahoo.com/video/apple-blowout-...
4 https://finance.yahoo.com/m/07524444-63a7-34aa...
5 https://finance.yahoo.com/news/etfs-stack-reop...
6 https://finance.yahoo.com/news/big-tech-stocks...
7 https://finance.yahoo.com/news/top-stock-repor...
8 https://finance.yahoo.com/news/why-stem-stock-...
9 https://finance.yahoo.com/video/technology-lar...

```

```

                                symbols \
0 AAPL.US, AAPL34.SA, AMZN.US, FB.US, GOOGL.US
1 AAPL.US, CCIV-UN.US, CCIV.US
2 AAPL.US, AAPL34.SA, AMZN.US, MSFT.US
3 AAPL.US, AAPL34.SA, BRK-A.US, BRK-B.US, DJI.IN...
4 AAPL.US, AAPL34.SA, SPOT.US
5 AAPL.US, AAPL34.SA, COMP.US, FB.US, GOOGL.US, ...
6 AAPL.US, CRM.US, FB.US, GME.US, GOOGL.US, MSFT...
7 AAPL.US, AAPL34.SA, BLK.US, F.US, GOOGL.US, ZT...
8 AAPL.US, AMZN.US, BX.US, FSLR.US, GE.US, GEOO3...
9 AAPL.US, AAPL34.SA, AMZN.US, DJI.INDX, FB.US, ...

```

```

                                tags sentiment_polarity \
0 AMAZON, APPLE, FACEBOOK, HARLEY FINKELSTEIN, M... 0.395
1 BUSINESS COMBINATION, CCIV, CHURCHILL CAPITAL ... 0.998
2 AMAZON, APPLE INC, BIG TECH, MARKET CAPITALIZA... 0
3 ALEXIS CHRISTOFOROUS, BERKSHIRE HATHAWAY, COWE... 0
4 APP STORE, APPLE, EUROPEAN COMMISSION, MUSIC S... -0.71
5 APPLE, CONSENSUS ESTIMATE, EARNINGS PER SHARE,... 0.999
6 BITCOIN, DIGITAL TRANSFORMATION, FACEBOOK, MIC... 0.999
7 ALPHABET, APPLE, APPLE WATCH, BLACKROCK, RESEA... 0.999
8 CLEAN-ENERGY, STEM STOCK 0.999
9 ALEXIS CHRISTOFOROUS, KEITH FITZ-GERALD, SIBIL... 0

```

```

sentiment_neg sentiment_neu sentiment_pos
0 0 0.871 0.129
1 0.023 0.85 0.127
2 0 1 0
3 0 1 0
4 0.093 0.885 0.022
5 0.027 0.826 0.148
6 0.026 0.861 0.113
7 0.042 0.775 0.183
8 0.015 0.83 0.154
9 0 1 0

```

```
[5]: # Sort raw_data by date column from earliest to latest
raw_data = raw_data.sort_values('date', ascending=True).reset_index(drop=True)

#raw_data.head(10)
raw_data.tail(10)
```

```
[5]:      date title content link symbols tags sentiment_polarity sentiment_neg \
35594  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35595  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35596  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35597  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35598  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35599  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35600  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35601  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35602  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN
35603  NaN   NaN     NaN  NaN      NaN  NaN                NaN          NaN

      sentiment_neu sentiment_pos
35594            NaN            NaN
35595            NaN            NaN
35596            NaN            NaN
35597            NaN            NaN
35598            NaN            NaN
35599            NaN            NaN
35600            NaN            NaN
35601            NaN            NaN
35602            NaN            NaN
35603            NaN            NaN
```

```
[6]: #check how many NA values there are. We care most about sentiment, date, and
      ↳ content columns
raw_data.isna().sum().sort_values(ascending=False)
```

```
[6]: tags                17790
sentiment_pos           5853
sentiment_neu           5812
sentiment_neg           5767
sentiment_polarity      5721
symbols                 5539
link                   5340
content                 4901
title                   4255
date                    2808
dtype: int64
```

```
[7]: #Drop Clumns we won't be focusing on (link and tags)
raw_data = raw_data.drop(columns=['link', 'tags'])
```

```
[8]: # force sentiment scores to be a numerical value, NA if not
for col in
    ['sentiment_pos', 'sentiment_neu', 'sentiment_neg', 'sentiment_polarity']:
    raw_data[col] = pd.to_numeric(raw_data[col], errors='coerce')

# extract valid calendar dates only (ignore time + BST noise)
raw_data['date'] = pd.to_datetime(
    raw_data['date'].astype(str).str.extract(r'(\d{4}-\d{2}-\d{2})')[0],
    errors='coerce'
) #This section of the code was inputed to handle warnings from pandas that
    could lead to future breakage of the code.

# convert content to string
raw_data['content'] = raw_data['content'].astype('str')
```

```
[9]: # Setting "date" column to datetime format
raw_data['date'] = pd.to_datetime(raw_data['date'])

print(raw_data.shape)

#Data range for raw_data
print(f"Date range for raw_data: {raw_data['date'].min()} to {raw_data['date'].
    max()}")
```

(35604, 8)

Date range for raw_data: 2016-02-19 00:00:00 to 2024-11-27 00:00:00

```
[10]: #this data is messy, drop observations that have NAs in critical columns
raw_data = raw_data.
    dropna(subset=['sentiment_pos', 'sentiment_neu', 'sentiment_neg', 'sentiment_polarity', 'content'])

#print out the data type of each column to ensure conversions from above worked
for col in raw_data.columns:
    print(f'Column: {col} , Data Type: {raw_data[col].dtype}')
```

```
Column: date , Data Type: datetime64[ns]
Column: title , Data Type: object
Column: content , Data Type: object
Column: symbols , Data Type: object
Column: sentiment_polarity , Data Type: float64
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
```

```
[11]: #Create a single sentiment score that we can use to determine if an article
      ↪contains more negative or positive sentiment
raw_data['Net Sentiment'] = raw_data['sentiment_pos'] -
      ↪raw_data['sentiment_neg']

#while not the main target variable, net sentiment could be interesting to
      ↪determine presence of positive and negative language
#we can also make this a classification model by coverting to 'Positive',
      ↪'Neutral', or 'Negative' here.
```

The “content” column will be the predictor variable in the first model, and the string will be transformed into a numerical matrix using feature engineering methods such as TF-IDF. The “snetiment polarity” variable will be the target variable. The goal of the first model will be to predict the sentiment polarity of a news article given its content.

```
[12]: print(raw_data.shape)
```

```
(29700, 9)
```

```
[13]: # Dropping rows before January 1, 2018 and after October 31, 2024
raw_data = raw_data[(raw_data['date'] >= '2018-01-01') & (raw_data['date'] <=
      ↪'2024-10-31')]

print(raw_data.shape)

#Data range for raw_data
print(f"Date range for raw_data: {raw_data['date'].min()} to {raw_data['date'].
      ↪max()}")
```

```
(29635, 9)
```

```
Date range for raw_data: 2018-01-31 00:00:00 to 2024-10-31 00:00:00
```

This dataset has 29,638 rows and 10 columns (plus the net sentiment score column created by us)

2.2 Price Timeseries Dataset

```
[14]: ts_data = pd.read_csv('../data/AAPL Stock Data.csv')
```

```
[15]: ts_data.head(10)
```

```
[15]:      Dates  AAPL Share Price
0  Jan-03-2005      1.13
1  Jan-04-2005      1.14
2  Jan-05-2005      1.15
3  Jan-06-2005      1.15
4  Jan-07-2005      1.24
5  Jan-10-2005      1.23
6  Jan-11-2005      1.15
7  Jan-12-2005      1.17
```

```
8 Jan-13-2005          1.25
9 Jan-14-2005          1.25
```

```
[16]: #check how many NA values there are
ts_data.isna().sum().sort_values(ascending=False)
```

```
[16]: Dates          0
AAPL Share Price    0
dtype: int64
```

```
[17]: #Convert values to the expected formatting
ts_data['Dates'] = pd.to_datetime(ts_data['Dates']).dt.date
ts_data['AAPL Share Price'] = ts_data['AAPL Share Price'].astype('float')
```

```
[18]: for col in ts_data.columns:
      print(f'Column: {col} , Data Type: {type(ts_data[col][0])}')

```

```
Column: Dates , Data Type: <class 'datetime.date'>
Column: AAPL Share Price , Data Type: <class 'numpy.float64'>
```

```
[19]: print(ts_data.shape)
```

```
(5298, 2)
```

```
[20]: # Setting "Dates" column to datetime format
ts_data['Dates'] = pd.to_datetime(ts_data['Dates'])

# Dropping rows before January 1, 2018 and after October 31, 2024
ts_data = ts_data[(ts_data['Dates'] >= '2018-01-01') & (ts_data['Dates'] <=
    ↪ '2024-10-31')]

print(ts_data.shape)
print(f"Date range: {ts_data['Dates'].min()} to {ts_data['Dates'].max()}")
```

```
(1720, 2)
```

```
Date range: 2018-01-02 00:00:00 to 2024-10-31 00:00:00
```

There are 1,720 rows in this dataset and two columns (date and price).

2.3 Combining Datasets

```
[21]: # Recreate sentiment_data from raw_data to ensure clean state
sentiment_data = raw_data.groupby(['date'])[['sentiment_polarity']].mean()

# Create a complete date range from January 1, 2018 to October 31, 2024
date_range = pd.date_range(start='2018-01-01', end='2024-10-31', freq='D')
complete_dates = pd.DataFrame({'date': date_range})

# Reset index to make 'date' a column
```

```

sentiment_data = sentiment_data.reset_index()

# Convert to datetime
sentiment_data['date'] = pd.to_datetime(sentiment_data['date'])

# Merge with complete date range
sentiment_data = pd.merge(complete_dates, sentiment_data, on='date', how='left')

# Set index back to date column
sentiment_data = sentiment_data.set_index('date')

print(f"Sentiment data shape after filling missing dates: {sentiment_data.
    ↪shape}")
sentiment_data.head(10)

```

Sentiment data shape after filling missing dates: (2496, 1)

```

[21]:          sentiment_polarity
date
2018-01-01          NaN
2018-01-02          NaN
2018-01-03          NaN
2018-01-04          NaN
2018-01-05          NaN
2018-01-06          NaN
2018-01-07          NaN
2018-01-08          NaN
2018-01-09          NaN
2018-01-10          NaN

```

```

[22]: #join on date
all_data = pd.merge(sentiment_data, ts_data, how = 'right', left_on = 'date',
    ↪right_on = 'Dates')

```

```

[23]: for col in all_data.columns:
        print(f'Column: {col} , Data Type: {type(all_data[col][0])}')

```

```

Column: sentiment_polarity , Data Type: <class 'numpy.float64'>
Column: Dates , Data Type: <class 'pandas._libs.tslibs.timestamps.Timestamp'>
Column: AAPL Share Price , Data Type: <class 'numpy.float64'>

```

```

[24]: all_data.head(10)

```

```

[24]:          sentiment_polarity      Dates  AAPL Share Price
0              NaN 2018-01-02          43.07
1              NaN 2018-01-03          43.06
2              NaN 2018-01-04          43.26
3              NaN 2018-01-05          43.75

```


4	NaN	2018-01-08	43.59
5	NaN	2018-01-09	43.58
6	NaN	2018-01-10	43.57
7	NaN	2018-01-11	43.82
8	NaN	2018-01-12	44.27
9	NaN	2018-01-16	44.05

We have a greater amount of stock price data than news sentiment data on the AAPL stock. There are currently 5,298 rows and 3 columns, but only 1,574 rows have sentiment data (as shown from the print statement above). We can remove dates without any sentiment analysis at a future date. For right now, this data could be helpful for a traditional forecasting model

```
[25]: all_data['sentiment_polarity'].describe()
```

```
[25]: count    1112.000000
      mean       0.531748
      std       0.299017
      min      -0.993000
      25%       0.412746
      50%       0.554172
      75%       0.685809
      max       1.000000
      Name: sentiment_polarity, dtype: float64
```

The sentiment polarity looks to be performing as expected. Scores range from -.99 to 1.00, but most are positive (which is expected due to Apple's stellar performance during this time period).

All models will use next-day stock price direction as the main target variable. There will be three possible forecasting models, the traditional model will exclusively use previous stock prices as predictor variables, the second model exclusively uses sentiment analysis as the predictor variable, while the third model will combine both.

```
[26]: #Data range for all_data
      print(f"Date range for all_data: {all_data['Dates'].min()} to_
      ↪{all_data['Dates'].max()}")
```

```
Date range for all_data: 2018-01-02 00:00:00 to 2024-10-31 00:00:00
```

3 Week 3 - Split and EDA

This week, we will focus on splitting the data into a train, validation, and test set and performing exploratory data analysis (EDA).

3.1 Split Sentiment Dataset

```
[27]: raw_data = raw_data.sort_values("date").reset_index(drop=True)

      raw_train = raw_data[(raw_data["date"] >= "2018-01-01") & (raw_data["date"] <=
      ↪"2022-10-31")]
```

```

raw_val = raw_data[(raw_data["date"] >= "2022-11-01") & (raw_data["date"] <=
↪"2023-10-31")]
raw_test = raw_data[(raw_data["date"] >= "2023-11-01") & (raw_data["date"] <=
↪"2024-10-31")]

print(raw_train.shape)
print(raw_val.shape)
print(raw_test.shape)

raw_train.head(10)

```

(15474, 9)

(7598, 9)

(6563, 9)

```

[27]:      date                                     title \
0 2018-01-31  Investor Expectations to Drive Momentum within...
1 2018-03-16  Top 100 Reputable Companies Around the Globe A...
2 2018-03-27  Universal Display Corporation Stock Is Way Und...
3 2018-04-16  Detailed Research: Economic Perspectives on Ge...
4 2018-06-22  Apple iPhone Spared Tariffs, But Could Face Ch...
5 2018-10-23  Report: Exploring Fundamental Drivers Behind T...
6 2018-11-30  New Research Coverage Highlights Omeros, Apple...
7 2019-01-03  Apple, AAPL Investment Losses Alert: Bernstein...
8 2019-01-03  SHAREHOLDER ALERT: Bronstein, Gewirtz & Grossm...
9 2019-02-22  Factors of Influence in 2019, Key Indicators a...

                                     content \
0  NEW YORK, Jan. 31, 2018 (GLOBE NEWSWIRE) -- ...
1  Boston, MA, March 16, 2018 (GLOBE NEWSWIRE) ...
2  Since topping out in January, shares of Univer...
3  NEW YORK, April 16, 2018 (GLOBE NEWSWIRE) --...
4  In an escalating trade war between China and t...
5  NEW YORK, Oct. 23, 2018 (GLOBE NEWSWIRE) -- ...
6  NEW YORK, Nov. 30, 2018 (GLOBE NEWSWIRE) -- ...
7  NEW YORK, Jan. 02, 2019 (GLOBE NEWSWIRE) -- ...
8  NEW YORK, Jan. 03, 2019 (GLOBE NEWSWIRE) -- ...
9  NEW YORK, Feb. 22, 2019 (GLOBE NEWSWIRE) -- ...

                                     symbols sentiment_polarity \
0          AAPL.US, AVHI.US, FARM.US, GM.US, SGRY.US          0.995
1  AAPL.US, AMZN.US, BA.US, BUD.US, CAJ.US, CAT.U...      0.991
2          AAPL.US, OLED.US          -0.244
3  AAPL.US, FB.US, GD.US, GRMN.US, HD.US, WBA.US          0.997
4          AAPL.US, HNHPF.US          -0.807
5  AAPL.US, BTX.US, CTLT.US, HD.US, PDLI.US, QURE.US          0.995
6  AAPL.US, BIO.US, COT.US, DAR.US, OMER.US, OR.US          0.998

```

7	AAPL.US	0.992
8	AAPL.US	0.967
9	AAPL.US, CBAY.US, FTK.US, GVA.US, IMGN.US, UAL.US	0.997

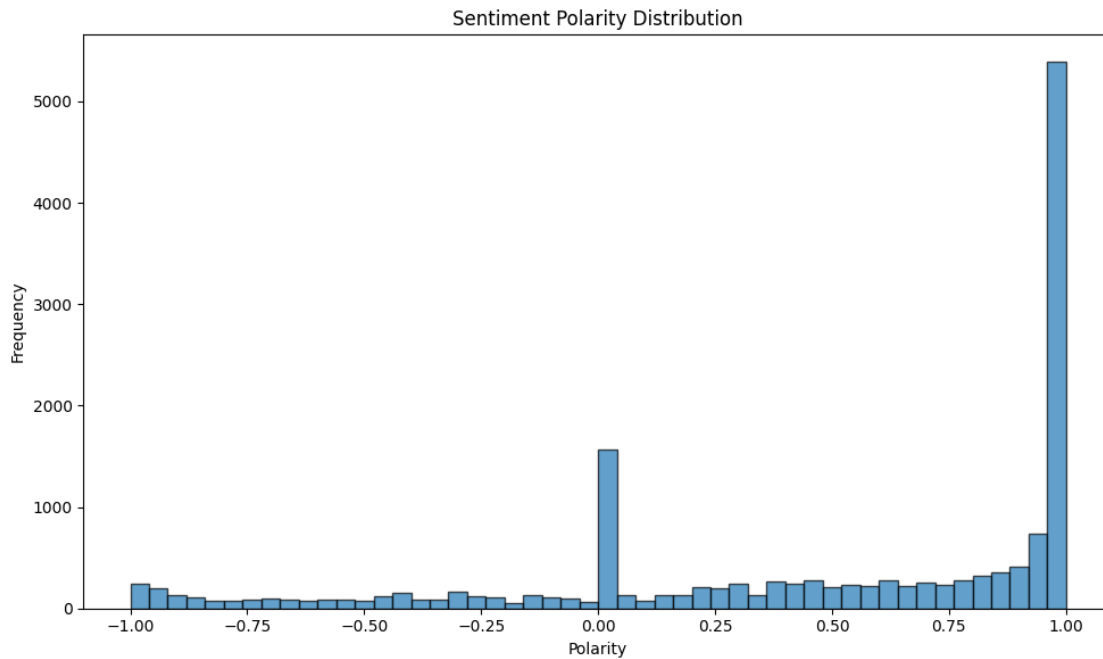
	sentiment_neg	sentiment_neu	sentiment_pos	Net Sentiment
0	0.009	0.937	0.054	0.045
1	0.011	0.917	0.072	0.061
2	0.134	0.764	0.101	-0.033
3	0.008	0.930	0.062	0.054
4	0.138	0.862	0.000	-0.138
5	0.008	0.941	0.051	0.043
6	0.009	0.925	0.066	0.057
7	0.038	0.816	0.147	0.109
8	0.050	0.839	0.111	0.061
9	0.009	0.928	0.064	0.055

3.2 EDA Sentiment Dataset

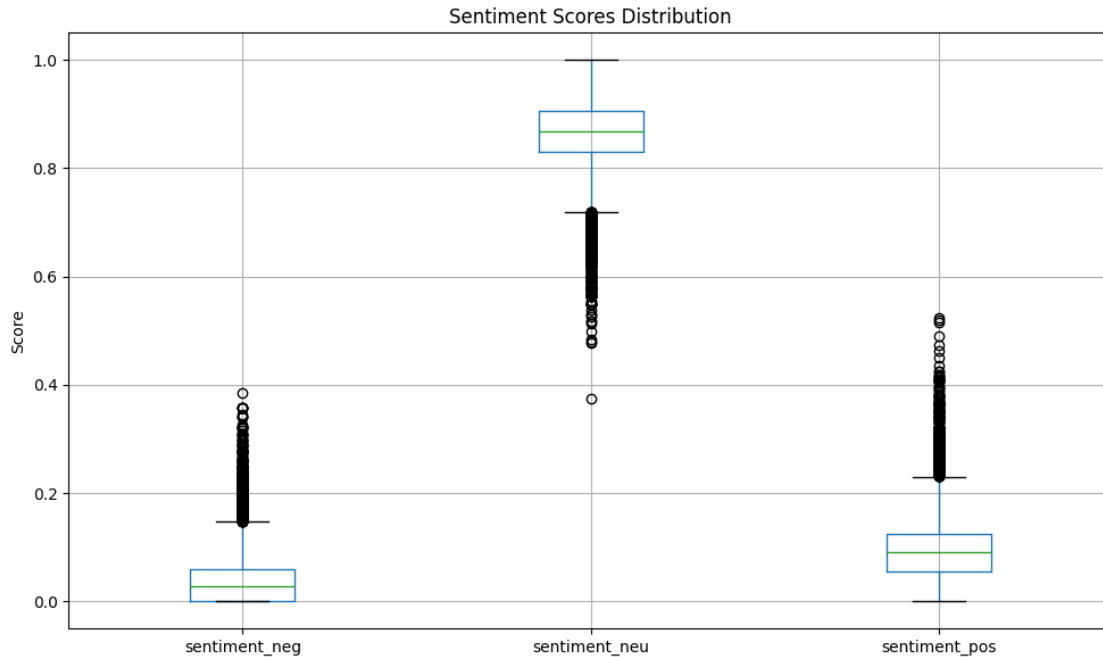
Visual Analysis:

- Sentiment polarity distribution histogram
- Sentiment scores comparison (neg/neu/pos) boxplot
- Articles count over time (monthly trend)
- Average sentiment polarity over time with zero baseline

```
[28]: # Sentiment polarity distribution
plt.figure(figsize=(10, 6))
plt.hist(raw_train['sentiment_polarity'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Sentiment Polarity Distribution')
plt.xlabel('Polarity')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



```
[29]: # Sentiment scores comparison
plt.figure(figsize=(10, 6))
sentiment_cols = ['sentiment_neg', 'sentiment_neu', 'sentiment_pos']
raw_train[sentiment_cols].boxplot()
plt.title('Sentiment Scores Distribution')
plt.ylabel('Score')
plt.tight_layout()
plt.show()
```

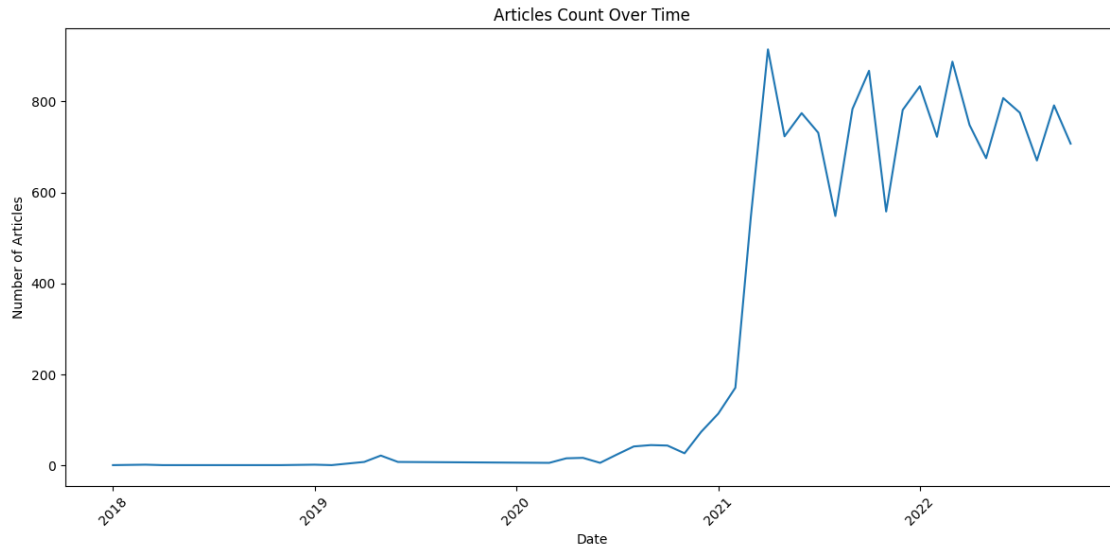


```
[30]: # Articles per date
raw_train['date'] = pd.to_datetime(raw_train['date'])
articles_per_date = raw_train.groupby(raw_train['date'].dt.to_period('M')).
    .size()
plt.figure(figsize=(12, 6))
plt.plot(articles_per_date.index.to_timestamp(), articles_per_date.values)
plt.title('Articles Count Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Articles')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

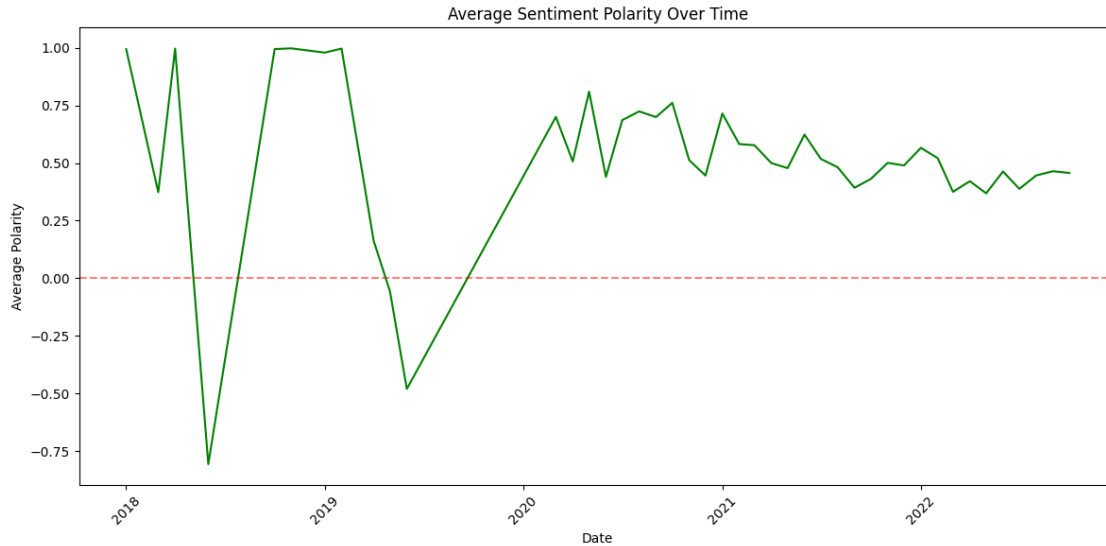
/tmp/ipykernel_1935268/3404307298.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
raw_train['date'] = pd.to_datetime(raw_train['date'])
```



```
[31]: # Average sentiment over time
avg_sentiment = raw_train.groupby(raw_train['date'].dt.
    ↳to_period('M'))['sentiment_polarity'].mean()
plt.figure(figsize=(12, 6))
plt.plot(avg_sentiment.index.to_timestamp(), avg_sentiment.values,
    ↳color='green')
plt.title('Average Sentiment Polarity Over Time')
plt.xlabel('Date')
plt.ylabel('Average Polarity')
plt.axhline(y=0, color='r', linestyle='--', alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



NLP Analysis on Title:

- Character length and word count distributions
- Average statistics

```
[32]: # Title length distribution
raw_train['title_length'] = raw_train['title'].fillna('').str.len()
      ↪ # Title word count

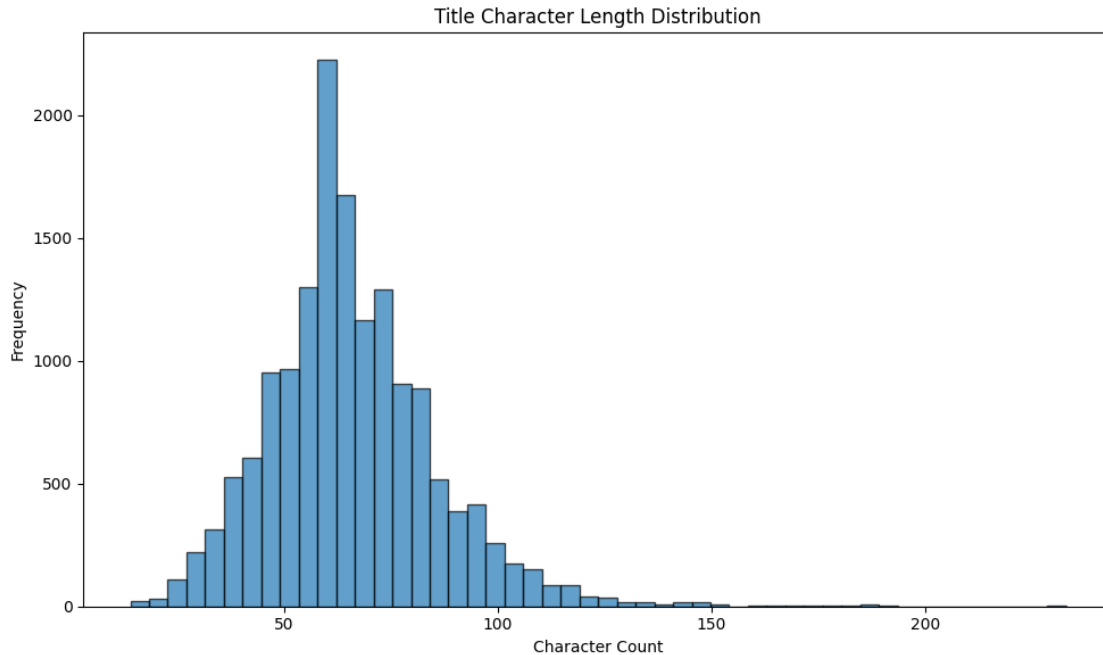
# Title character length
plt.figure(figsize=(10, 6))
plt.hist(raw_train['title_length'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Title Character Length Distribution')
plt.xlabel('Character Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"\nAverage title length: {raw_train['title_length'].mean():.2f}
      ↪characters")
```

/tmp/ipykernel_1935268/1666406537.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
raw_train['title_length'] = raw_train['title'].fillna('').str.len()
# Title word count
```



Average title length: 65.83 characters

```
[33]: # Title word count
raw_train['title_word_count'] = raw_train['title'].fillna('').str.split().str.
    ↪len()

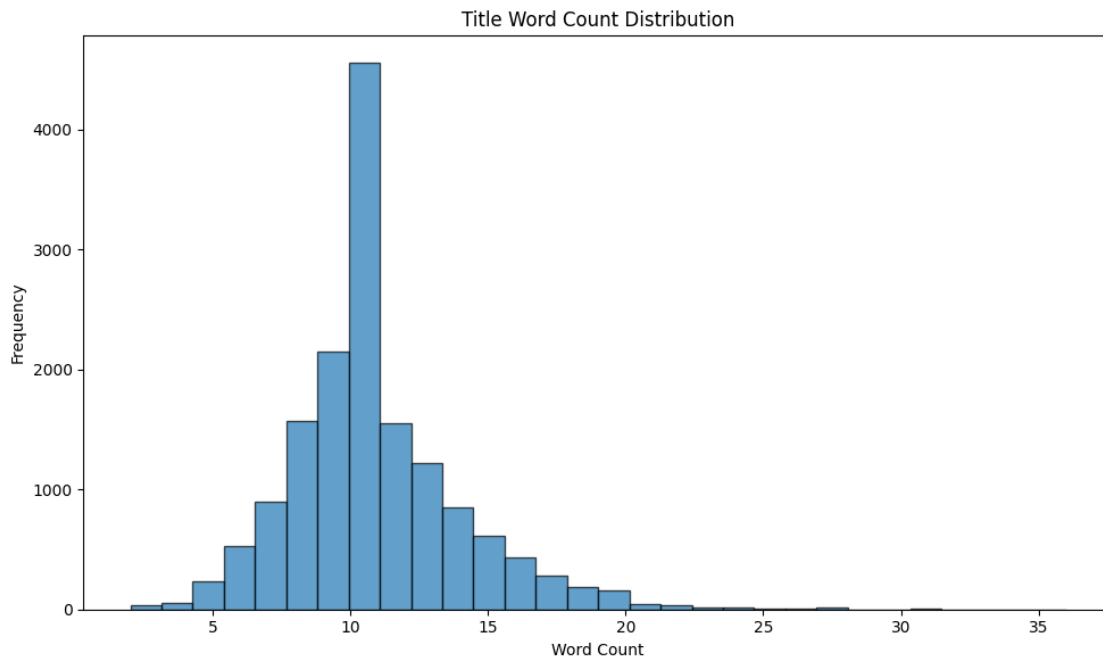
# Title word count
plt.figure(figsize=(10, 6))
plt.hist(raw_train['title_word_count'], bins=30, edgecolor='black', alpha=0.7)
plt.title('Title Word Count Distribution')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"Average title word count: {raw_train['title_word_count'].mean():.2f}␣
    ↪words")
```

/tmp/ipykernel_1935268/4066365091.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
raw_train['title_word_count'] =


```
raw_train['title'].fillna('').str.split().str.len()
```



Average title word count: 10.85 words

- Top 20 most common words
- NLP Analysis on Content:

Character length and word count distributions Average statistics and missing content count Correlation Analysis:

Sentiment vs title/content length scatter plots

```
[34]: # Most common words in titles
all_title_words = ' '.join(
    raw_train['title']
    .fillna('')
    .astype(str) # AI Added code to help prevent code breakage in future due
    ↳to warning errors.
).lower()

words = re.findall(r'\b[a-z]+\b', all_title_words)
word_freq = Counter(words)
common_words = word_freq.most_common(20)

print("\nTop 20 most common words in titles:")
for word, count in common_words:
    print(f" {word}: {count}")
```

Top 20 most common words in titles:

```
apple: 5885
to: 5033
s: 3821
stocks: 2929
the: 2874
in: 2615
stock: 2316
and: 1933
on: 1873
as: 1798
for: 1730
buy: 1617
of: 1600
a: 1587
is: 1494
tech: 1490
market: 1359
dow: 1321
earnings: 1098
with: 973
```

```
[35]: # Content length distribution
raw_train.loc[:, 'content_length'] = (
    raw_train['content']
    .fillna('')
    .astype(str)
    .str.len()
) # AI Added code to help prevent code breakage in future due to warning errors.

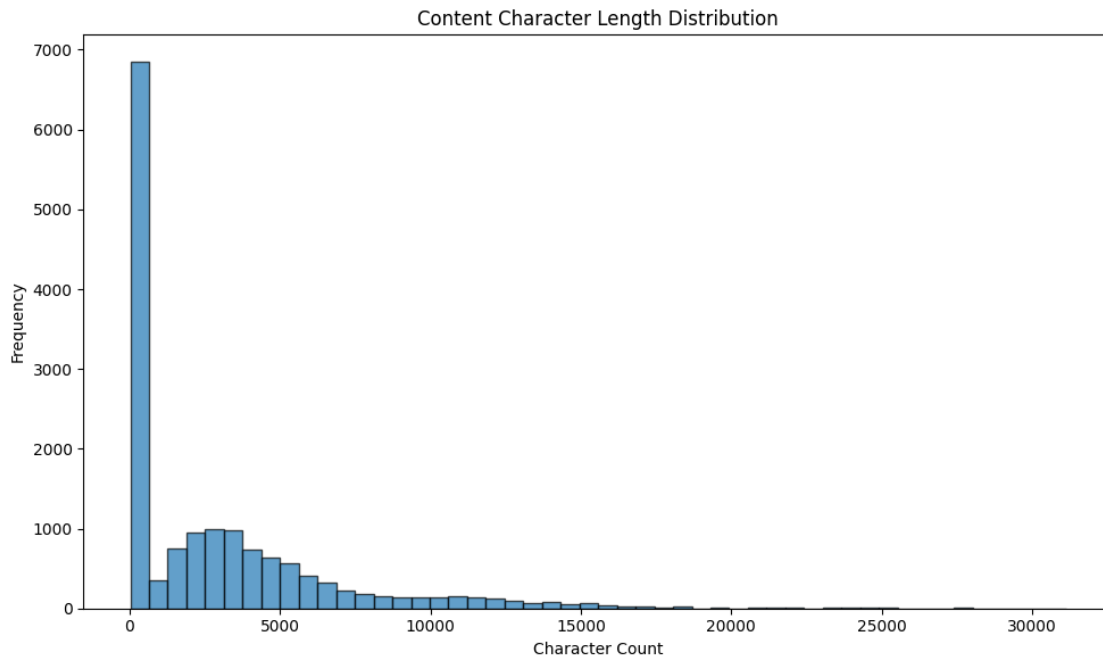
# Content character length
plt.figure(figsize=(10, 6))
plt.hist(raw_train['content_length'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Content Character Length Distribution')
plt.xlabel('Character Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"\nAverage content length: {raw_train['content_length'].mean():.2f}␣
↪characters")
```

/tmp/ipykernel_1935268/2720773849.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: <https://pandas.pydata.org/pandas->

```
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
raw_train.loc[:, 'content_length'] = (
```



Average content length: 3020.90 characters

```
[36]: # Content word count
raw_train.loc[:, 'content_word_count'] = (
    raw_train['content']
    .fillna('')
    .astype(str)
    .str.split()
    .str.len()
) # AI Added code to help prevent code breakage in future due to warning errors.

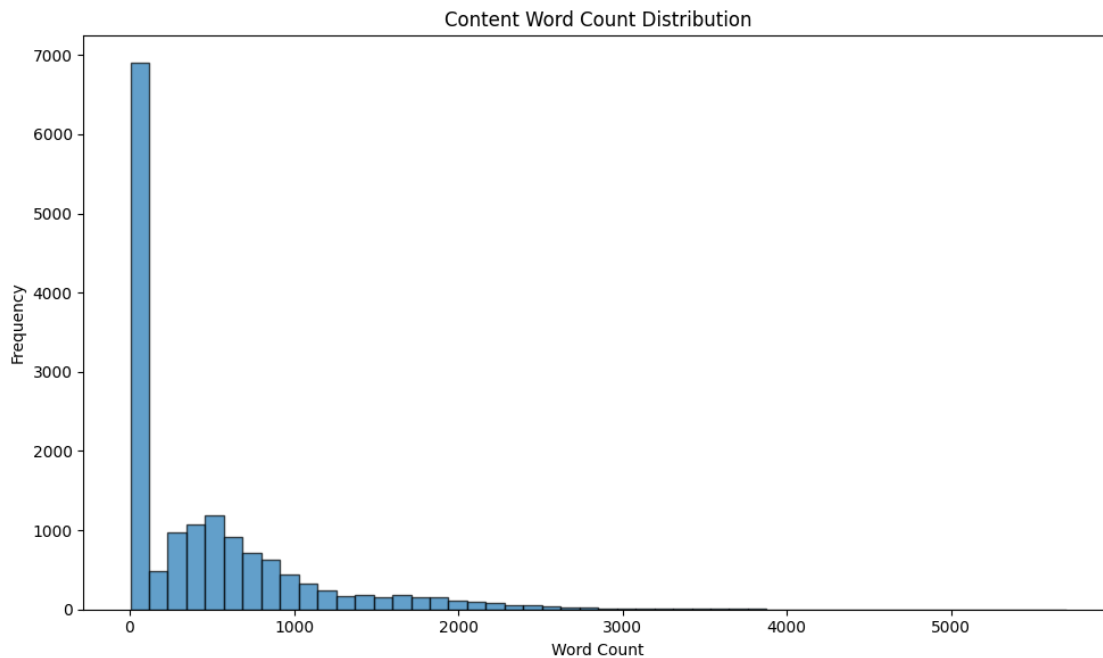
# Content word count distribution
plt.figure(figsize=(10, 6))
plt.hist(raw_train['content_word_count'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Content Word Count Distribution')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"Average content word count: {raw_train['content_word_count'].mean():.
↪2f} words")
```

```
/tmp/ipykernel_1935268/233806758.py:2: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
raw_train.loc[:, 'content_word_count'] = (
```

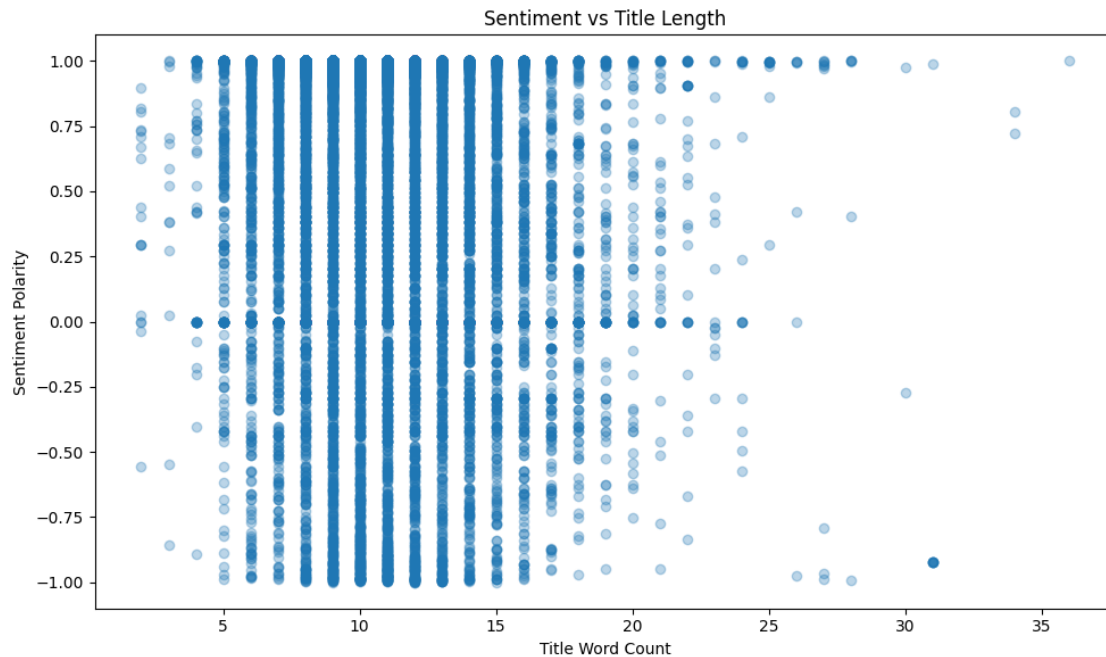


Average content word count: 481.55 words

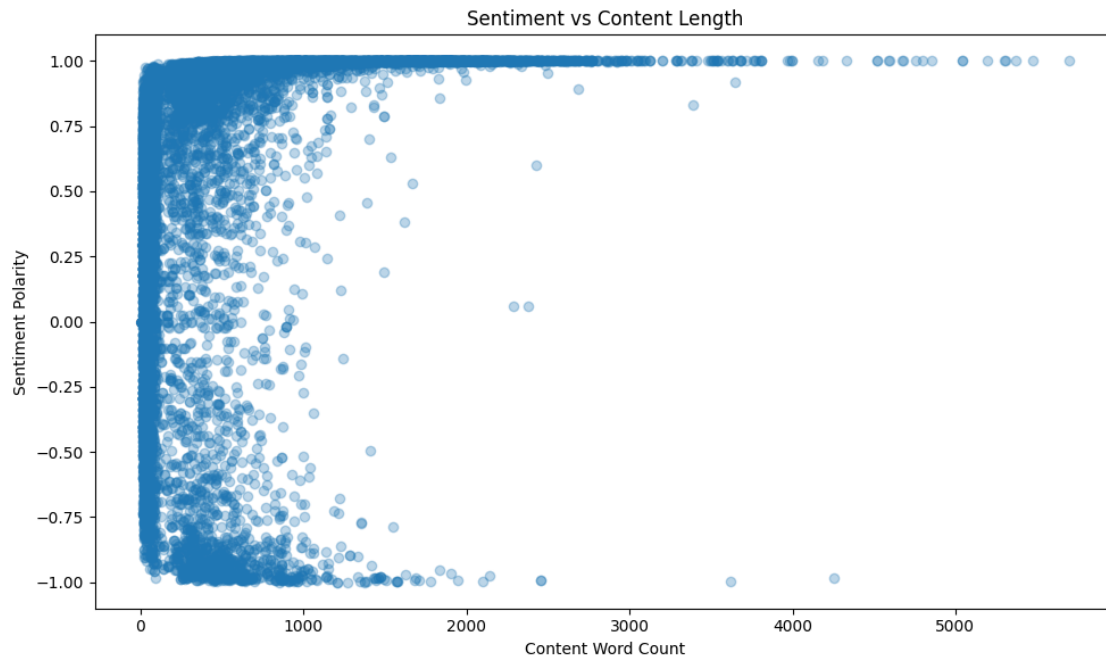
```
[37]: print(f"Missing content: {raw_train['content'].isna().sum()} articles")
```

Missing content: 0 articles

```
[38]: # Sentiment vs Title Length  
plt.figure(figsize=(10, 6))  
plt.scatter(raw_train['title_word_count'], raw_train['sentiment_polarity'],  
            alpha=0.3)  
plt.title('Sentiment vs Title Length')  
plt.xlabel('Title Word Count')  
plt.ylabel('Sentiment Polarity')  
plt.tight_layout()  
plt.show()
```



```
[39]: # Sentiment vs Content Length
plt.figure(figsize=(10, 6))
plt.scatter(raw_train['content_word_count'], raw_train['sentiment_polarity'],
            alpha=0.3)
plt.title('Sentiment vs Content Length')
plt.xlabel('Content Word Count')
plt.ylabel('Sentiment Polarity')
plt.tight_layout()
plt.show()
```



3.3 Split Price Dataset

```
[40]: ts_data = ts_data.sort_values("Dates").reset_index(drop=True)

ts_train = ts_data[(ts_data["Dates"] >= "2018-01-01") & (ts_data["Dates"] <=
↪ "2022-10-31")]
ts_val   = ts_data[(ts_data["Dates"] >= "2022-11-01") & (ts_data["Dates"] <=
↪ "2023-10-31")]
ts_test  = ts_data[(ts_data["Dates"] >= "2023-11-01") & (ts_data["Dates"] <=
↪ "2024-10-31")]

print(ts_train.shape)
print(ts_val.shape)
print(ts_test.shape)

ts_train.head(10)
```

```
(1217, 2)
```

```
(251, 2)
```

```
(252, 2)
```

```
[40]:      Dates  AAPL Share Price
0 2018-01-02      43.07
1 2018-01-03      43.06
2 2018-01-04      43.26
```

3	2018-01-05	43.75
4	2018-01-08	43.59
5	2018-01-09	43.58
6	2018-01-10	43.57
7	2018-01-11	43.82
8	2018-01-12	44.27
9	2018-01-16	44.05

3.4 EDA Price Dataset

```
[41]: ts_train.describe()
```

```
[41]:
```

	Dates	AAPL Share Price
count	1217	1217.000000
mean	2020-06-01 17:34:15.875102720	96.583131
min	2018-01-02 00:00:00	35.550000
25%	2019-03-20 00:00:00	50.690000
50%	2020-06-03 00:00:00	81.300000
75%	2021-08-17 00:00:00	142.000000
max	2022-10-31 00:00:00	182.010000
std	NaN	46.350782

```
[42]: # Check for missing values
print("Missing values in ts_train:")
print(ts_train.isnull().sum())
print(f"\nPercentage of missing values:")
print((ts_train.isnull().sum() / len(ts_train) * 100).round(2))
```

Missing values in ts_train:

```
Dates      0
AAPL Share Price  0
dtype: int64
```

Percentage of missing values:

```
Dates      0.0
AAPL Share Price  0.0
dtype: float64
```

```
[43]: # Time Series Plot of Close Price
plt.figure(figsize=(15, 6))
plt.plot(ts_train['Dates'], ts_train['AAPL Share Price'], linewidth=1.5,
         color='blue')
plt.title('Stock Close Price Over Time (Training Data)', fontsize=16,
         fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Close Price ($)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
```

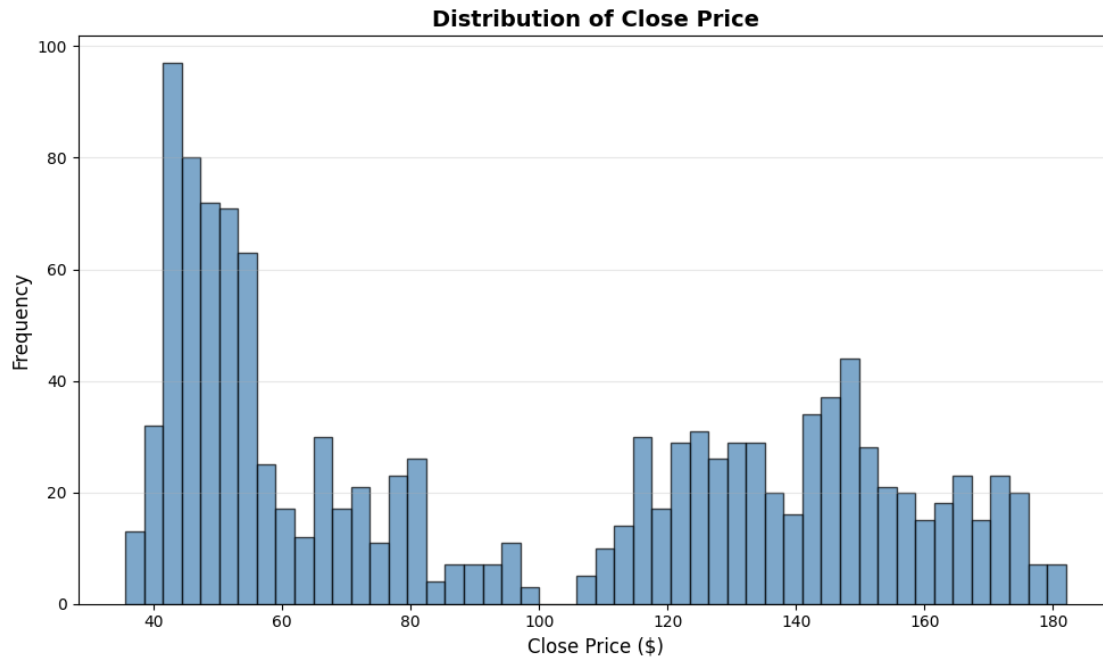
```
plt.show()
```



```
[44]: # Distribution of Close Price
plt.figure(figsize=(10, 6))

# Histogram
plt.hist(ts_train['AAPL Share Price'], bins=50, color='steelblue',
        edgecolor='black', alpha=0.7)
plt.title('Distribution of Close Price', fontsize=14, fontweight='bold')
plt.xlabel('Close Price ($)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```

```
[45]: # STL Decomposition (Seasonal-Trend decomposition using LOESS)
# Prepare time series for STL
ts_for_stl = ts_train.set_index('Dates')['AAPL Share Price']

# Perform STL decomposition
stl = STL(ts_for_stl, seasonal=13, period=252) # 252 trading days per year
stl_result = stl.fit()

# Plot decomposition
fig, axes = plt.subplots(4, 1, figsize=(15, 12))

# Original
axes[0].plot(stl_result.observed, color='blue', linewidth=1)
axes[0].set_ylabel('Observed', fontsize=12)
axes[0].set_title('STL Decomposition of Close Price', fontsize=16,
                  fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Trend
axes[1].plot(stl_result.trend, color='green', linewidth=1.5)
axes[1].set_ylabel('Trend', fontsize=12)
axes[1].grid(True, alpha=0.3)

# Seasonal
axes[2].plot(stl_result.seasonal, color='orange', linewidth=1)
```

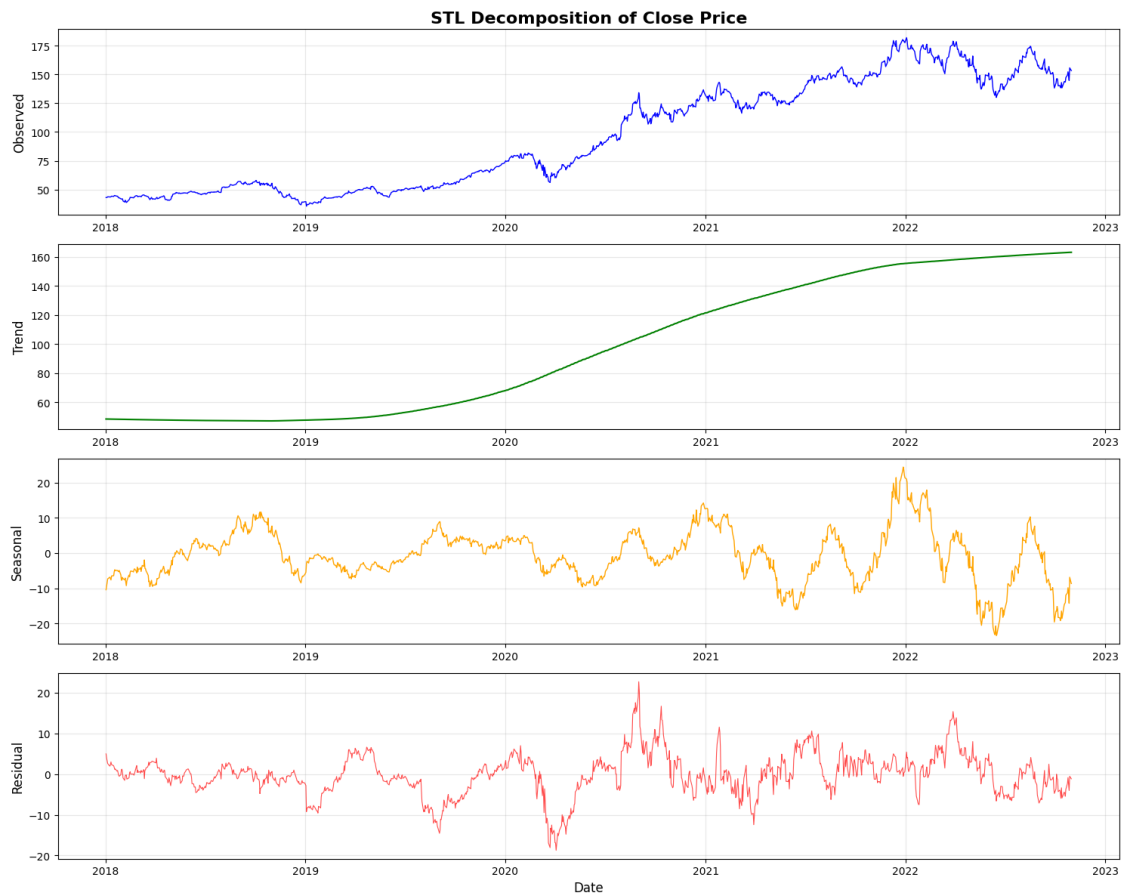
```

axes[2].set_ylabel('Seasonal', fontsize=12)
axes[2].grid(True, alpha=0.3)

# Residual
axes[3].plot(stl_result.resid, color='red', linewidth=0.8, alpha=0.7)
axes[3].set_ylabel('Residual', fontsize=12)
axes[3].set_xlabel('Date', fontsize=12)
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```

[46]: # Rolling Statistics (Moving Average and Standard Deviation)
window_sizes = [7, 30, 90] # 1 week, 1 month, 3 months

fig, axes = plt.subplots(2, 1, figsize=(15, 10))

# Rolling Mean

```

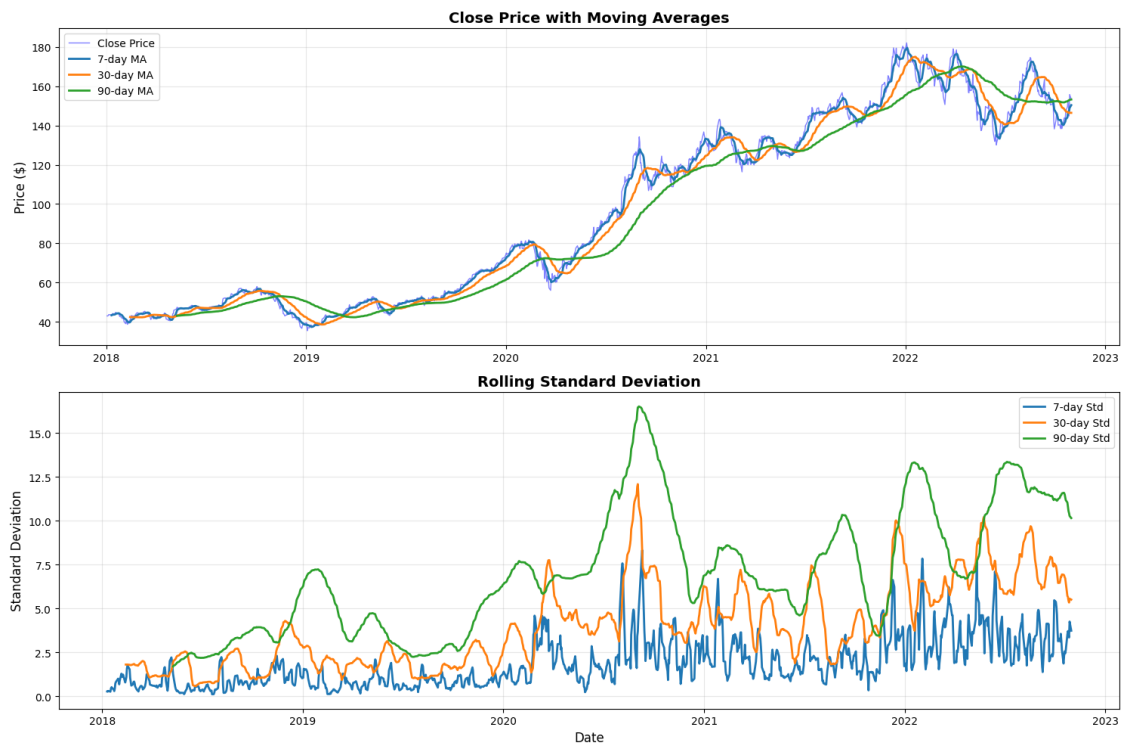
```

axes[0].plot(ts_train['Dates'], ts_train['AAPL Share Price'], label='Close Price', linewidth=1, alpha=0.5, color='blue')
for window in window_sizes:
    rolling_mean = ts_train['AAPL Share Price'].rolling(window=window).mean()
    axes[0].plot(ts_train['Dates'], rolling_mean, label=f'{window}-day MA', linewidth=2)
axes[0].set_title('Close Price with Moving Averages', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Price ($)', fontsize=12)
axes[0].legend(loc='best')
axes[0].grid(True, alpha=0.3)

# Rolling Standard Deviation
for window in window_sizes:
    rolling_std = ts_train['AAPL Share Price'].rolling(window=window).std()
    axes[1].plot(ts_train['Dates'], rolling_std, label=f'{window}-day Std', linewidth=2)
axes[1].set_title('Rolling Standard Deviation', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Standard Deviation', fontsize=12)
axes[1].legend(loc='best')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

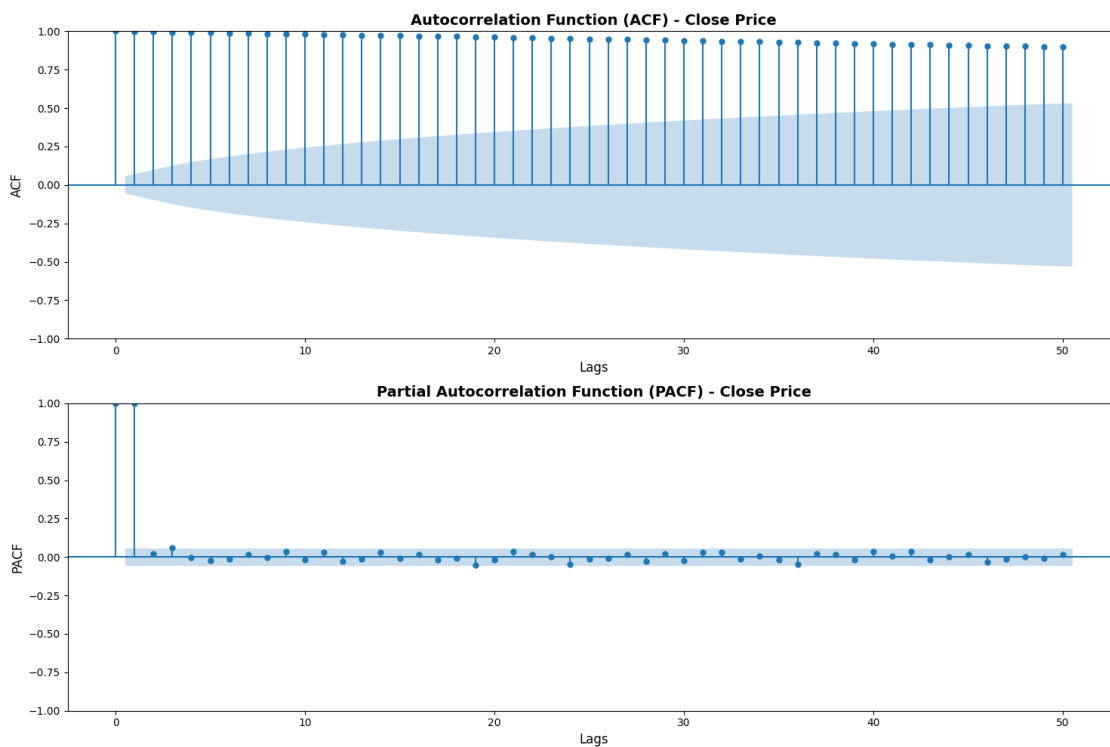


```
[47]: # ACF and PACF Plots
fig, axes = plt.subplots(2, 1, figsize=(15, 10))

# ACF
plot_acf(ts_train['AAPL Share Price'].dropna(), lags=50, ax=axes[0])
axes[0].set_title('Autocorrelation Function (ACF) - Close Price', fontsize=14,
    fontweight='bold')
axes[0].set_xlabel('Lags', fontsize=12)
axes[0].set_ylabel('ACF', fontsize=12)

# PACF
plot_pacf(ts_train['AAPL Share Price'].dropna(), lags=50, ax=axes[1])
axes[1].set_title('Partial Autocorrelation Function (PACF) - Close Price',
    fontsize=14, fontweight='bold')
axes[1].set_xlabel('Lags', fontsize=12)
axes[1].set_ylabel('PACF', fontsize=12)

plt.tight_layout()
plt.show()
```



3.5 Split Combined Dataset

```
[48]: all_data["Dates"] = pd.to_datetime(all_data["Dates"])
all_data = all_data.sort_values("Dates").reset_index(drop=True)

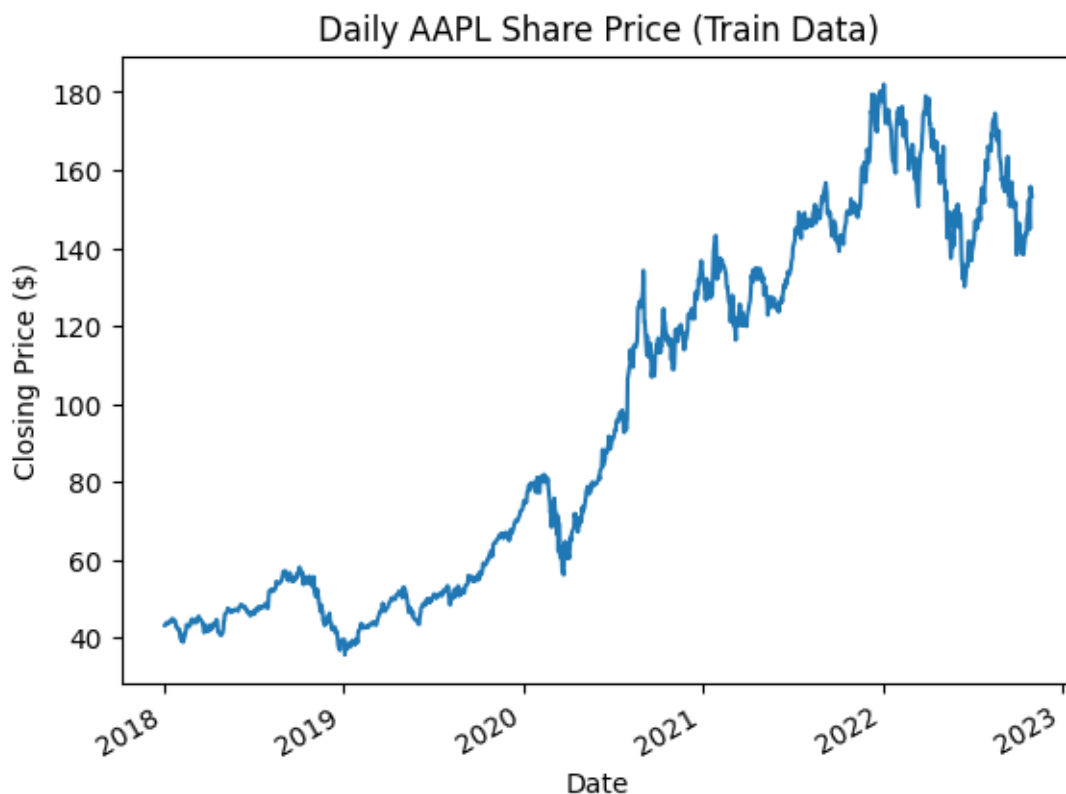
com_train = all_data[(all_data["Dates"] >= "2018-01-01") & (all_data["Dates"]_
↵<= "2022-10-31")]
com_val    = all_data[(all_data["Dates"] >= "2022-11-01") & (all_data["Dates"]_
↵<= "2023-10-31")]
com_test   = all_data[(all_data["Dates"] >= "2023-11-01") & (all_data["Dates"]_
↵<= "2024-10-31")]
```

Now, we have three separate sets of data. To prevent data leakage, all data processing will be done to each set separately. Next, we will perform EDA on only the training data.

First, we build a plot using only the time series stock price data.

3.6 EDA Combined Dataset (Training Data)

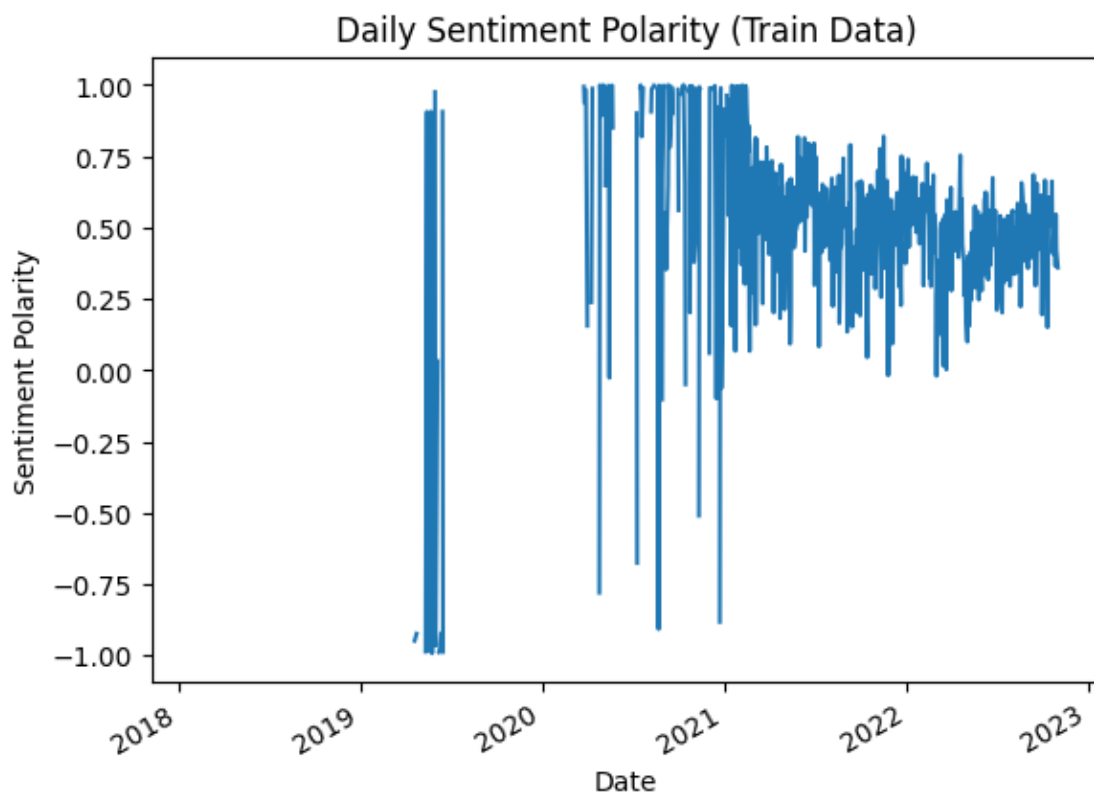
```
[49]: plt.plot(com_train['Dates'], com_train['AAPL Share Price'])
plt.xlabel('Date')
plt.ylabel('Closing Price ($)')
plt.title('Daily AAPL Share Price (Train Data)')
plt.gcf().autofmt_xdate()
plt.show()
```



During the period of time covered by the train data, AAPL stock fluctuates, but increases overall from July, 2020 to about January 2022. Then, it continues to fluctuate but trends down until July, 2022.

We can also build a plot of the sentiment polarity over the same period of time and compare the two charts.

```
[50]: plt.plot(com_train['Dates'], com_train['sentiment_polarity'])
plt.xlabel('Date')
plt.ylabel('Sentiment Polarity')
plt.title('Daily Sentiment Polarity (Train Data)')
plt.gcf().autofmt_xdate()
plt.show()
```



The sentiment polarity plot shows many missing values at the start of the time series. Then, there are some extreme highs and lows. Many of these are close to 1 (maximum) and -1 (minimum) and a few are exactly 1.

There are some low outliers in 2020. These could be explained by the COVID-19 pandemic, which brought uncertainty to the stock market and the economy as a whole.

Most of the sentiment scores, especially in the latter half, are overall positive.

Also, since we know that sentiment polarity scores in this dataset are all in the -1 to 1 range, we can confirm that there are no illogical values in the train data by looking at this plot.

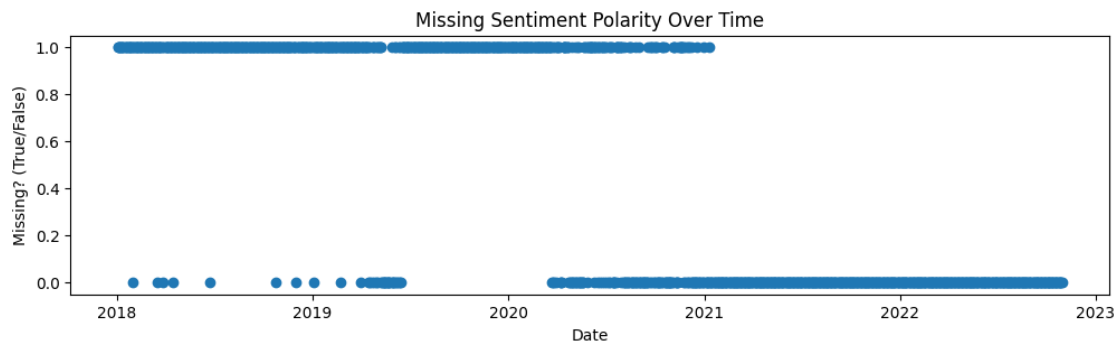
Next, we can examine the extent of missing values, specifically within the sentiment polarity feature.

```
[51]: com_train['sentiment_polarity'].isna().sum()
```

```
[51]: np.int64(608)
```

Overall, there are 608 missing values for sentiment polarity within the train data and no missing values for AAPL stock price. Next, we can visualize where exactly these missing values occur in the time series data.

```
[52]: plt.figure(figsize=(12,3))
plt.plot(
    com_train['Dates'],
    com_train['sentiment_polarity'].isna(),
    marker='o',
    linestyle='None'
)
plt.title("Missing Sentiment Polarity Over Time")
plt.xlabel("Date")
plt.ylabel("Missing? (True/False)")
plt.show()
```

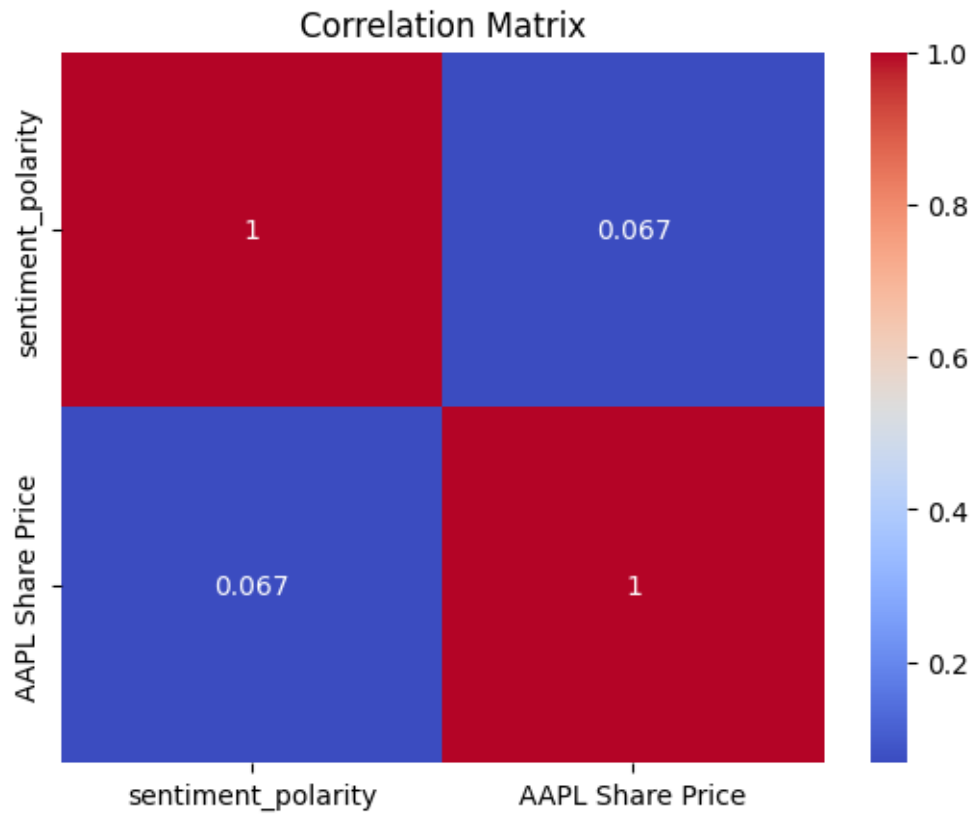


As expected from the previous plot, most of the 608 missing values occur in the earlier months of the train data.

We can also view a correlation matrix to look at how our two variables are correlated.

```
[53]: corr_matrix = com_train[["sentiment_polarity", "AAPL Share Price"]].corr()

sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Finally, we can observe a seasonal composition of the train data.

```
[54]: y = com_train['sentiment_polarity'].dropna()

decomp = seasonal_decompose(y, model='additive', period=7)
decomp.plot()
plt.gcf().set_size_inches(8, 5)
plt.show()
```